

1.2 — Orchestrate Multi-Agent Systems with Coordinator-Subagent Patterns

Concept

Multi-agent systems follow a **hub-and-spoke** (coordinator-subagent) pattern. A single coordinator agent receives a complex task, decomposes it into well-scoped subtasks, delegates each subtask to a specialized subagent, and then aggregates the results into a final response. The coordinator never tries to do everything itself — it drives decisions, while each subagent operates within guardrails enforced in code.

The critical design principle is **isolated context**: every subagent runs in its own independent conversation. It has no memory of what other subagents said, and it cannot read the coordinator's full conversation history. This isolation prevents context contamination, keeps each subagent focused, and makes the system easier to reason about. The coordinator is the only node that holds the full picture — it receives outputs from each spoke and synthesizes them.

Task decomposition is where the model earns its role. The coordinator's system prompt instructs Claude to analyze the incoming request and produce a structured breakdown of subtasks before any subagent is spawned. Each subtask has a clear scope, a defined output format, and a minimal `allowedTools` list. Result aggregation happens after all subagents complete: the coordinator receives their outputs as tool results, reasons across them, and produces a coherent final answer. The mental model is: **the model drives decisions, code enforces guardrails** — Claude decides which subagents to invoke and how to interpret their outputs; your application code controls which tools each subagent may access and what data it can see.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// Coordinator: receives a complex task and delegates to subagents
async function coordinator(userTask: string): Promise<string> {
  const messages: Anthropic.MessageParam[] = [
    { role: "user", content: userTask },
  ];

  // Agentic loop – coordinator decides which subagents to invoke
  while (true) {
    const response = await client.messages.create({
```

```

model: "claude-opus-4-5",
max_tokens: 4096,
system:
  "You are a coordinator. Break complex tasks into subtasks and delegate each to a special agent."
tools: [
  {
    name: "run_research_agent",
    description: "Spawn a subagent that researches a specific topic.",
    input_schema: {
      type: "object" as const,
      properties: {
        query: {
          type: "string",
          description: "The focused research question.",
        },
      },
      required: ["query"],
    },
  },
  {
    name: "run_analysis_agent",
    description: "Spawn a subagent that analyzes provided data.",
    input_schema: {
      type: "object" as const,
      properties: {
        data: { type: "string", description: "Raw data to analyze." },
      },
      required: ["data"],
    },
  },
],
messages,
});

if (response.stop_reason === "end_turn") {
  const textBlock = response.content.find((b) => b.type === "text");
  return textBlock ? textBlock.text : "";
}

// Execute tool calls – each spawns an isolated subagent conversation
const toolResults: Anthropic.ToolResultBlockParam[] = [];
for (const block of response.content) {
  if (block.type !== "tool_use") continue;

  let result: string;
  if (block.name === "run_research_agent") {
    const input = block.input as { query: string };

```

```

        result = await researchSubagent(input.query); // isolated context
    } else {
        const input = block.input as { data: string };
        result = await analysisSubagent(input.data); // isolated context
    }

    toolResults.push({ type: "tool_result", tool_use_id: block.id, content: result });
}

messages.push({ role: "assistant", content: response.content });
messages.push({ role: "user", content: toolResults });
}
}

// Subagent: isolated conversation, restricted tools
async function researchSubagent(query: string): Promise<string> {
    const response = await client.messages.create({
        model: "claude-haiku-4-5",
        max_tokens: 1024,
        system: "You are a research specialist. Answer only the question asked.",
        messages: [{ role: "user", content: query }],
        // No tools – this subagent only reasons, does not act
    });

    const textBlock = response.content.find((b) => b.type === "text");
    return textBlock?.text ?? "";
}

async function analysisSubagent(data: string): Promise<string> {
    const response = await client.messages.create({
        model: "claude-haiku-4-5",
        max_tokens: 1024,
        system: "You are a data analyst. Summarize key insights from the data.",
        messages: [{ role: "user", content: `Analyze this data:\n\n${data}` }],
    });

    const textBlock = response.content.find((b) => b.type === "text");
    return textBlock?.text ?? "";
}

```

Common Mistakes

1. **Sharing the full conversation history with subagents** — Each subagent should receive only the context it needs for its specific subtask. Passing the coordinator's entire message history bloats the context window, leaks unrelated information, and produces worse-focused outputs.

2. **Letting subagents call other subagents directly** — In a hub-and-spoke topology, only the coordinator routes work. Allowing subagents to spawn their own subagents creates uncontrolled fan-out, makes aggregation logic unpredictable, and breaks the single-point-of-synthesis guarantee.
3. **Skipping task decomposition and going straight to subagent invocation** — Without a structured decomposition step, the coordinator tends to spawn overlapping or redundant subagents. Always instruct the coordinator to plan subtasks first; the decomposition itself is where the model earns its role.

References

- [Anthropic: Build multi-agent systems](#)
- [Anthropic: Tool use overview](#)
- [Anthropic: Model overview \(choosing Haiku vs Sonnet vs Opus\)](#)